

Atelier en opti avancée Projet - p2

Antonin Sylvestre
Thomas Blanché

April 2026

Table des matières

1	Introduction	2
2	Principes de l'Optimisation Boîte Noire [1]	2
2.1	Le Concept de "Black Box"	2
2.2	Absence de Dérivées	2
2.3	Défis du Monde Réel	2
3	NOMAD et l'Algorithme MADS [1]	3
3.1	L'Algorithme MADS	3
3.2	Gestion des Paramètres de Grille	3
3.3	NOMAD	3
4	Notre implémentation de NOMAD	4
4.1	Nomad en Mode Batch	4
4.2	NOMAD en Librairie	4
5	Résultats Et Discussions	5
5.1	Résultat avec NOMAD en mode batch	6
5.2	Résultat avec NOMAD en mode librairie	7
5.3	Comparaison sur les 100 premières lignes	8
5.4	Discussion et perspective	9
5.4.1	Difficultés rencontrée	9
5.4.2	Amélioration des fonctionnalités	9
6	Conclusion	10
7	Annexe	11
7.1	Code Informatique	11
7.2	Bibliographie	12

1 Introduction

Dans le cadre du cours "Atelier en optimisation avancée - Hiver 2026", nous réalisons un projet visant à mettre en place une solution calculant la répartition optimale d'eau dans un barrage hydroélectrique à l'aide de divers algorithmes comme de la programmation Dynamique, traitée dans notre rapport précédent [2]. Ce rapport rend compte de la deuxième partie de ce projet.

Dans cette deuxième partie, l'objectif est de mettre en place un système donnant la répartition optimale de l'eau dans une centrale hydroélectrique. Cela dans l'objectif de maximiser la production électrique avec de l'optimisation par Boîte Noire, et plus précisément l'algorithme MADS, en passant par NOMAD. Nous pourrions ensuite comparer les résultats avec ceux obtenus dans la première partie du projet (par programmation dynamique). Les paramètres permettant le calcul sont les mêmes que dans la première partie : le débit total à répartir et les bornes inférieure et supérieure sur le débit acceptable par chacune des turbines, ainsi que la hauteur d'eau en amont.

L'étape suivante dans le cadre de ce projet sera de créer une interface, qui a déjà été commencé. Nous avons donc du adapter nos programmes pour convenir à cette contrainte.

2 Principes de l'Optimisation Boîte Noire [1]

L'optimisation boîte noire s'inscrit dans le domaine plus large de l'optimisation non-linéaire. Elle est privilégiée lorsque l'on cherche à minimiser une fonction objectif $f(x)$ dont la structure interne est inconnue ou inaccessible.

2.1 Le Concept de "Black Box"

Dans ce paradigme, la fonction est traitée comme un système fermé :

- On fournit un vecteur de variables d'entrée x au système.
- Le système retourne une valeur de sortie $f(x)$.
- Le processus interne reste opaque pour l'algorithme d'optimisation.

2.2 Absence de Dérivées

La caractéristique fondamentale de ces problèmes est l'impossibilité d'accéder aux dérivées ou au gradient de la fonction (∇f). L'optimisation doit donc se baser uniquement sur les valeurs de la fonction pour diriger la recherche.

2.3 Défis du Monde Réel

L'optimisation boîte noire doit souvent faire face à des contraintes :

- **Coût de calcul** : Chaque évaluation (appel à la boîte noire) peut prendre beaucoup de temps.

- **Bruit** : Les résultats peuvent être sujets à des imprécisions numériques ou du "bruit".
- **Échecs de simulation** : Pour certains x , la boîte noire peut simplement ne pas retourner de valeur. Il est donc toujours important de prendre en compte les bornes de son problème, si elles sont connues.

3 NOMAD et l'Algorithme MADS [\[1\]](#)

Pour résoudre ces problèmes, le logiciel **NOMAD** implémente l'algorithme **MADS** (Mesh Adaptive Direct Search).

3.1 L'Algorithme MADS

MADS est un algorithme itératif de recherche directe qui ne nécessite pas d'information sur le gradient. Il fonctionne sur une grille de recherche appelée "Mesh". Chaque itération se décompose généralement en deux étapes :

1. **L'étape SEARCH** : Une recherche globale est effectuée sur le Mesh pour tenter de trouver un point qui améliore l'objectif. Cette étape est flexible et peut utiliser diverses heuristiques.
2. **L'étape POLL** : Si l'étape Search échoue, une recherche locale plus rigoureuse est effectuée autour du meilleur point actuel (le "succès").

3.2 Gestion des Paramètres de Grille

La précision de la recherche est contrôlée par deux paramètres de taille :

- Δ_k^m (Mesh size) : Définit la finesse de la grille de calcul.
- Δ_k^p (Poll size) : Définit la distance maximale à laquelle on peut sonder de nouveaux points autour de la solution actuelle.

3.3 NOMAD

NOMAD est l'outil logiciel (développé en C++) qui permet d'appliquer concrètement l'algorithme MADS à des problèmes industriels. Il est conçu pour être robuste, gérer les contraintes de manière efficace (via des barrières progressives ou extrêmes) et minimiser le nombre d'évaluations de la boîte noire

4 Notre implémentation de NOMAD

Modélisation Mathématique de notre problème :

Variables : $x = [Q_1, Q_2, Q_3, Q_4, Q_5]$ (les débits).

Objectif : Maximiser $f = \sum P_i(Q_i, H)$, vu par NOMAD comme minimiser $f(Q_1, Q_2, Q_3, Q_4, Q_5)$

Contraintes : $\sum Q_i \leq Q_{total}$ et $Q_{min} \leq Q_i \leq Q_{max}$.

4.1 Nomad en Mode Batch

La figure 1 est une représentation schématique du fonctionnement de NOMAD en mode batch.

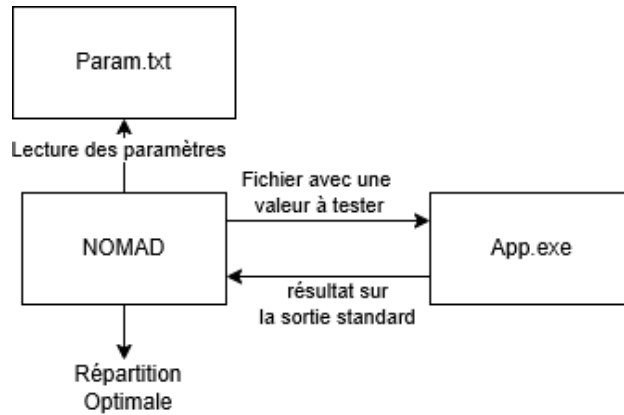


FIGURE 1 – Schéma du fonctionnement de NOMAD en mode batch

Ici, **App.exe** est notre exécutable, qui contient la boîte noire (notre centrale électrique). Elle est paramétrée par **NOMAD** au travers d'un fichier qu'il génère, et qui lui permet d'envoyer des paramètres, le débit alloué à chaque turbine. **NOMAD** est paramétré par un fichier **.txt** qui contient la définition de la forme et des bornes du problème, ainsi que d'autres paramètres. La solution trouvée est donnée directement dans le terminal.

Le problème de cette implémentation, c'est que la transformer en application portable et partageable est assez complexe, sans parler de la performance qui est moyenne. Nous l'avons tout de même testée et présentons les résultats dans la partie 5.1.

4.2 NOMAD en Librairie

La figure 2 est une représentation schématique du fonctionnement de **NOMAD** en mode librairie. Il faut noter que le fonctionnement réel de l'application est significativement plus complexe, puisque celle ci prend en compte la possibilité d'avoir N solveurs, ainsi qu'une interface qui permet de les utiliser, et doit donc s'adapter en conséquence.

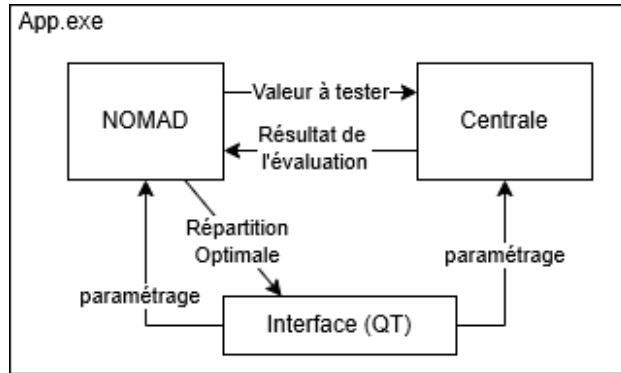


FIGURE 2 – Schéma de notre application

La différence principale avec le mode batch est qu'ici NOMAD est partie intégrante de l'application, et est utilisé au même titre que n'importe quelle autre librairie c++. Ainsi, le paramétrage de NOMAD est fait directement dans l'application, et NOMAD peut directement évaluer la boîte noire au sein de l'application, ce qui améliore significativement la vitesse d'exécution. Pour plus de détail consultez le diagramme de classe : figure 7, ou le code source [3].

5 Résultats Et Discussions

Afin de pouvoir tester notre projet plus rapidement, nous avons mis en place une interface, visible sur la figure 3. Celle-ci permet de configurer la majorité des variables de nos problèmes, et ce de manière plutôt compacte (les équations de productions n'y sont pas paramétrable). elle est également prévue pour être souple, et facilement modifiable si il fallait rajouter des turbines et/ou des algorithmes.

Turbine	Active	Min	Max	Débit optimisé	Puissance
Turbine 1	☒	0	160	0.00 m³/s	0.00 MW
Turbine 2	☒	0	160	147.00 m³/s	45.79 MW
Turbine 3	☒	0	160	139.00 m³/s	41.76 MW
Turbine 4	☒	0	160	148.00 m³/s	47.15 MW
Turbine 5	☒	0	160	144.00 m³/s	45.51 MW

Débit consommé: 578.00 m³/s Puissance totale: 180.21 MW

FIGURE 3 – Interface de l'application

5.1 Résultat avec NOMAD en mode batch

En utilisant les paramètres suivants :

```

1 BB_EXE          NomadExec.exe
2 DIMENSION       5
3 BB_INPUT_TYPE   ( R R R R R )
4 BB_OUTPUT_TYPE  OBJ EB
5
6 # Bornes unitaires (0 a 160 m3/s par turbine)
7 LOWER_BOUND     ( 0 0 0 0 0 )
8 UPPER_BOUND     ( 160 160 160 160 160 )
9
10 # Point de depart
11 X0              ( 1 1 1 1 1 )
12
13 # Limite d'evaluations
14 MAX_BB_EVAL     500
15
16 # Parametres d'affichage et de statistiques
17 DISPLAY_DEGREE  2
18 DISPLAY_STATS   BBE ( SOL ) OBJ
19 STATS_FILE      test.txt BBE ( SOL ) OBJ

```

Listing 1 – Fichier de paramètres param.txt pour l'optimisation NOMAD

Et en prenant pour référence la première ligne de donnée du tableur excell, avec un débit total de 578.0 m^3 et une hauteur amont de 137.88 m , nous obtenons le déroulé visible dans la Table 1. Ce que l'on constate avec le Tableau 1, c'est que les 3 résultats sont très proche. À noter : la résolution de la programmation dynamique était de $1 \text{ m}^3/\text{s}$. Après 500 étapes qui prennent 17 secondes à être calculées, NOMAD ne fait jamais de choix qui varie de plus de $1 \text{ m}^3/\text{s}$ par rapport à la programmation dynamique. Cela nous permet de valider le bon fonctionnement de NOMAD.

On constate que dans les premières itérations (jusqu'à la 27^{ième}), la croissance est relati-

TABLEAU 1 – Comparaison de la progression NOMAD avec les données réelles et la programmation dynamique

Source / BBE	Q_1	Q_2	Q_3	Q_4	Q_5	P_{tot} (MW)
1	1.0000	1.0000	1.0000	1.0000	1.0000	0.000
2	1.0000	11.0000	11.0000	0.0000	21.0000	7.244
6	0.0000	160.000	160.000	80.000	160.000	164.632
27	0.0000	140.000	150.000	120.000	160.000	175.041
93	0.0000	150.000	147.000	130.000	150.000	178.761
158	0.6000	145.900	142.900	145.800	142.800	179.984
268	0.0000	146.800	139.900	149.300	142.000	180.192
350	0.0020	147.646	138.187	148.667	143.492	180.204
NOMAD (500)	0.0001	147.719	138.006	148.723	143.551	180.206
Reel (Ligne 1)	158.0	141.0	0.0	140.0	140.0	179.2
Calcul DP	0.0	147.0	139.0	148.0	144.0	180.3

vement forte. Après, elle ralentit énormément. Le temps pris par les itérations 158 à 500 (environ 11.5 secondes, les 2/3 du temps de calcul) ne permettent d'augmenter la valeur du résultat que d'environ 0.1 %. Cela permettrait tout de même de fournir 80 maison québécoises en électricité si l'on considère que la consommation est toujours égale à la consommation moyenne [4].

5.2 Résultat avec NOMAD en mode librairie

FIGURE 4 – Paramétrage de NOMAD en mode librairie (EB)

Le paramétrage de NOMAD en mode librairie a été fait comme montré dans la figure 4. A noter que pour le pas de recherche initial (qui règle la taille du mesh), le réglage par défaut est plus proche de 6.5 avec les bornes de notre problème, mais ne donne pas de meilleurs résultats. Nous avons également testé NOMAD en mode librairie avec une barrière progressive, en ne changeant que le type de barrière. Les résultats suivant utilisent les mêmes paramètres physiques que précédemment : débit total de 578.0 m^3 et une hauteur amont de 137.88 m

On constate que les résultats de NOMAD en mode librairie sont significativement moins bon qu'en mode batch. Malgré le temps passé à jouer avec les différents paramètre, comme la MESH_UPDATE_BASIS qui définit le coefficient de descente, le pas d'arrêt, la taille initiale du mesh, le point de départ, que ce soit avec une barrière EB ou PB... Impossible de trouver une combinaison gagnante.

TABLEAU 2 – Seconde comparaison de NOMAD

Source / BBE	Q_1	Q_2	Q_3	Q_4	Q_5	P_{tot} (MW)
Reel (Ligne 1)	158.0	141.0	0.0	140.0	140.0	179.2
Calcul DP	0.0	147.0	139.0	148.0	144.0	180.3
NOMAD batch	0.0001	147.719	138.006	148.723	143.551	180.206
NOMAD lib EB	0	144	150	124	160	177.7
NOMAD lib PB	0	126	158	138	156	177.55

Les résultats trouvés par NOMAD en mode librairie sont environ 1.5% moins bons que ceux trouvés par NOMAD en mode batch. Cela dit, le temps d'exécution est significativement plus faible : 500 itérations se font en environ une seconde.

5.3 Comparaison sur les 100 premières lignes

En prenant les 100 premières lignes de données de la centrale, on peut comparer le temps d'exécution de chaque algorithme (figure 6) et la production prédite (ainsi que celle de la centrale, figure 5).

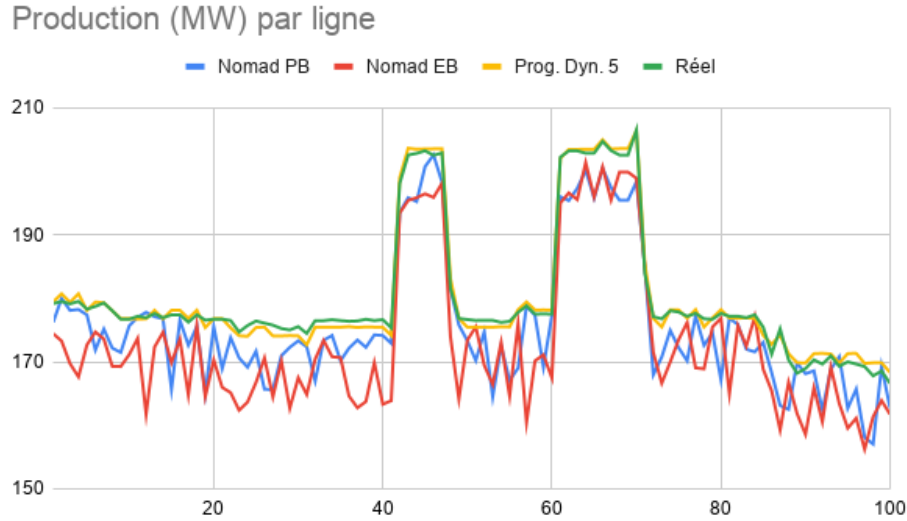


FIGURE 5 – Production de la centrale en fonction de l'algorithme utilisé

Si on regarde les valeurs moyennes, que l'on regarde le pourcentage de production en fonction de la valeur de la centrale et que l'on inscrit le temps moyen pour chaque calcul, voici ce que l'on obtient :

Algo.	Nomad EB	Nomad PB	DP - $5m^3/s$	DP - $0,05m^3/s$	Réelle
Prod. relative	96,06	97,61	100,02	100,42	100
Temps (s)	0,495	1,317	0,001	0,078	

Pour ce qui est de NOMAD, entre EB et PB, on constate que la version PB donne de meilleurs résultats, 1.6 point de plus, mais que le calcul est 2.7 fois plus long. Et dans tous

Temps d'exécution en seconde par ligne

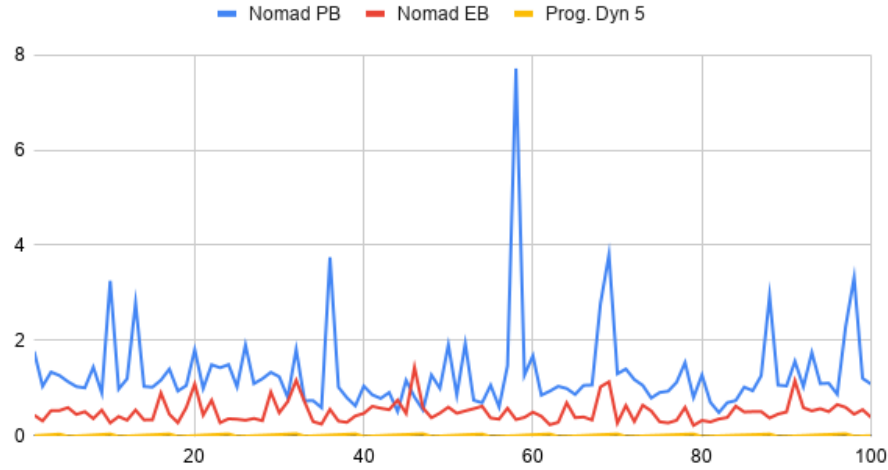


FIGURE 6 – Temps d'exécution de chaque algorithme

les cas, les résultats sont inférieurs à la production réelle de la centrale.

Le contraste est frappant avec l'algorithme de production Dynamique : en un temps entre 6 et 500 fois plus court (si on ne compare les temps qu'avec Nomad EB), on obtient un résultat toujours légèrement meilleur : La différence DP5 - Nomad PB est de 3.4 points, et DP0.05 - PB de 3.8 points.

5.4 Discussion et perspective

5.4.1 Difficultés rencontrées

Au départ, nous voulions implémenter un algorithme qui aurait fait une première passe avec la programmation dynamique, et utiliser le résultat en tant que point de départ pour les recherches de NOMAD afin de trouver une solution locale plus précise, mieux optimisée. Cela n'a pas été fait en raison du manque de précision de NOMAD : En mode librairie, nous n'avons pas réussi à trouver de paramètre qui permettait d'améliorer les résultats d'une programmation dynamique avec un pas de $5m^3/s$.

5.4.2 Amélioration des fonctionnalités

Dans l'hypothèse où notre application devrait pouvoir accommoder n'importe quelle centrale, il faudrait ajouter plusieurs fonctionnalités

- Rendre paramétrable le nombre de turbine.
- Rendre paramétrable toutes les équations de la centrale, que ce soit pour les turbines ou le calcul du niveau aval. Le plus simple serait de passer par un fichier de configuration textuel qui permettrait le copié collé depuis matlab.

Il y a également d'autres améliorations que l'application pourrait recevoir

- donner le contrôle total sur NOMAD : Il n'y a que quelques paramètres que l'utilisateur peut changer, alors que NOMAD en a beaucoup.
- Finition de l'interface : Certains éléments pourraient être mieux interconnectés (le choix du point initial de NOMAD avec les bornes des turbines par exemple)
- Intégration dans un écosystème : Quand la centrale reçoit l'ordre "traite $550m^3/s$ ", un système efficace proposerait directement une (ou plusieurs) solution. Notre application n'est absolument pas pensée dans cette optique.
- Exportation des résultats : Actuellement, comme montré dans la figure 3 la répartition du débit est uniquement affichée dans l'interface. Dans la vraie vie, il faudrait un moyen pour envoyer cette information jusqu'au vannes qui contrôle la centrale.

6 Conclusion

Notre Objectif dans le cadre de ce projet était d'appliquer MADS au travers de NOMAD afin d'optimiser la production d'une centrale hydroélectrique. En utilisant notre modèle de centrale comme une boîte noire, nous souhaitons comparer ces nouveaux résultats avec ceux de la programmation dynamiques établis préalablement [2].

Le constat est sans appel : pour ce problème spécifique, la programmation dynamique donne des résultats significativement meilleurs en un temps très significativement inférieur. Concernant NOMAD, nous avons découvert une étonnante différence entre les résultats du mode batch et du mode librairie. Que cette différence s'explique par une configuration suboptimale de notre part ou par une faiblesse inhérente au modèle reste à déterminer.

L'environnement physique modélisé n'est pas très complexe. Son comportement est prévisible. On peut supposer que NOMAD a été conçu pour fonctionner dans des environnements plus complexe, fortement bruités et/ou non linéaires. Notre démonstration ne lui a peut être pas vraiment permis d'exploiter toute ses capacités.

Dans tout les cas, au travers de l'application développée dans le cadre de ce projet, un utilisateur peut choisir d'utiliser NOMAD ou la programmation dynamique pour optimiser la production de sa centrale hydroélectrique. Il en tirera des scénarios de répartition robustes et exploitables. Cerise sur le gâteau, l'interface intègre une gestion dynamique du mode sombre pour garantir une expérience utilisateur fluide et moderne, quelle que soit l'heure du calcul.

7 Annexe

7.1 Code Informatique

Le code est trop long pour être inclus. Vous le trouverez ici : [3]

Voici tout de même le diagramme de classe du projet 7

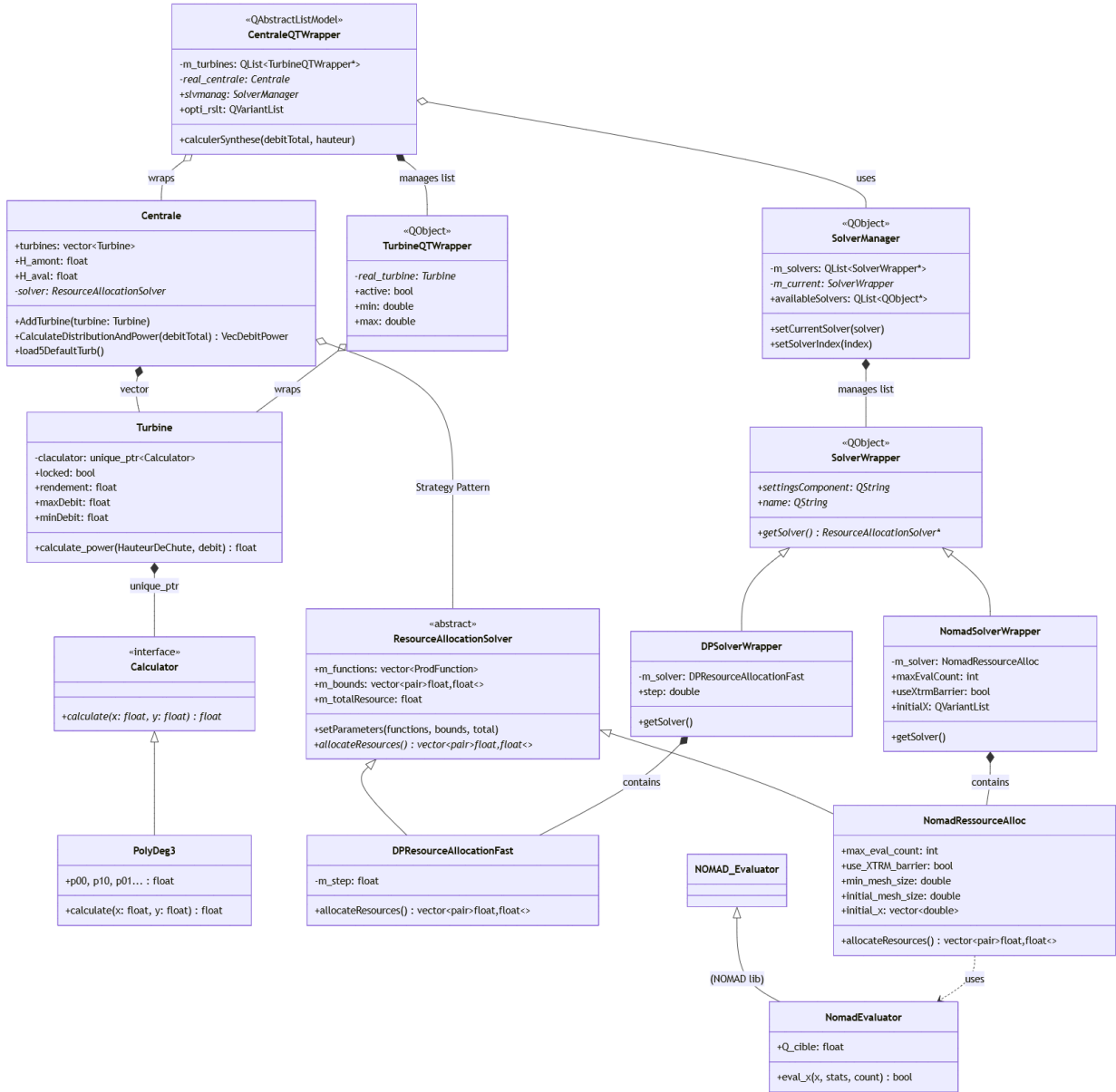


FIGURE 7 – Diagramme de classe du projet

à noter : Avec QT viennent des fichiers supplémentaires, des .qml qui permettent de décrire l'interface.

7.2 Bibliographie

Références

- [1] Sara Séguin. *Cours 8 - Optimisation non-linéaire et boîte noire*. Université du Québec à Chicoutimi (UQAC), 2026. Support de cours pour l’Atelier en optimisation avancée (8INF926).
- [2] Antonin Sylvestre & Thomas Blanché. Atelier en opti avancée : Projet 1 – modélisation et programmation dynamique. *Atelier en opti avancée Hiver UQAC*, february 2026. Rendu dans le cadre du cursus académique.
- [3] Antonin Sylvestre & Thomas Blanché. UQAC_OPTI : Outil d’optimisation de répartition de ressources hydroélectriques via nomad et qt. https://github.com/antonin-slv/UQAC_OPTI, 2026. Consulté le 12 avril 2026.
- [4] Statistique Canada. Tableau 25-10-0061-01 : Consommation d’énergie des ménages, par type de logement, canada et les provinces. Statistique Canada, avr 2024. Données spécifiques au Québec pour l’électricité (période 2019-2021). Consulté le 6 avril 2026. URL : <https://www150.statcan.gc.ca/t1/tbl1/fr/tv.action?pid=2510006101>, doi:10.25318/2510006101-fra.